

CSP-Bender Problem

Formulation, Solution in AMPL, and Comparison with Heuristic Methods

Liliu Martinez

Enrique Takiguchi

Contents

1	Introduction	2
2	Problem Description and Formulation	2
2.1	Basic definitions	2
2.2	CSP Formulation	2
2.3	Benders decomposition	3
3	Datasets	4
4	Implementation Details	4
5	References	4
A	AMPL Files	5

1 Introduction

Objectives:

- TODO

2 Problem Description and Formulation

2.1 Basic definitions

- **Microdata:** a file V of s individuals and t variables, as a matrix $s \times t$ where V_{ij} is the value of variable j for individual i .
- **Tabular data:** tables generated crossing one or more categorical variables from microdata. These can be multidimensional tables holding frequency or magnitude information. The table will have n cells which will have to verify m linear constraints. The cells in the table are bounded by an upper and lower limit: $u < a_i < l$
- **Protection levels from the p% rule:**

An attacker might estimate the intervals of a cell $[\bar{a}_p, \underline{a}_p]$, to assert these are not informative enough, we must define some lower and upper protection level bounds to ensure this interval fully contains the interval

$$[\bar{a}_p, \underline{a}_p] \in [a_p - lpl_p, a_p + upl_p] \quad (1)$$

2.2 CSP Formulation

The Cell Suppression Problem seeks to hide sensitive (primary, $\mathcal{P}$) cells while minimizing the suppression cost. Hiding only primary cells would not be enough since we could still infer information about them from the rest of cells, thus, a number of secondary cells ($\mathcal{S}$) will also be suppressed.

CSP computes the set $\mathcal{S}$ of secondary cells to be suppressed such that for all $p \in \mathcal{P}$, Equation 1 is satisfied, where $\bar{a}_p, \underline{a}_p$ can be obtained with a minimization problem.

The CSP problem can be formulated as a Mixed-Integer Linear Programming (MILP) problem. The MILP will be a minimization problem in terms of the following variables

- $y_i \in \{0, 1\}, i = 1, \dots, n$: Indicates if a cell i is suppressed (1) or not (0)
- $x^{l,p} \in \mathbb{R}^n$: Lower deviation of the primary cell p from its original value
- $x^{u,p} \in \mathbb{R}^n$: Upper deviation of the primary cell p from its original value

MILP formulation for CSP:

$$\begin{aligned} \min \quad & \sum_{i=1}^n w_i y_i \\ \text{s.to} \quad & \left. \begin{aligned} & Ax^{l,p} = 0 \\ (l_i - a_i)y_i \leq & \begin{cases} x_i^{l,p} \leq (u_i - a_i)y_i \\ x_p^{l,p} \leq -lpl_p \end{cases} & i = 1, \dots, n \end{aligned} \right\} \forall p \in \mathcal{P} \\ & \left. \begin{aligned} & Ax^{u,p} = 0 \\ (l_i - a_i)y_i \leq & \begin{cases} x_i^{u,p} \leq (u_i - a_i)y_i \\ x_p^{u,p} \geq upl_p \end{cases} & i = 1, \dots, n \end{aligned} \right\} \\ & y_i \in \{0, 1\} \quad x \geq 0 \quad i = 1, \dots, n, \end{aligned}$$

The problem with the MILP formulation is that it becomes infeasibly complex as the table grows large.

The problem has:

- n binary variables and $2n\mathcal{P}$ continuous variables
- $2(m + 2n)|\mathcal{P}|$ constraints

2.3 Benders decomposition

Benders decomposition partitions the problem into a master and subproblems, and iterates efficiently between them to find the optimal solution.

- Master problem: focuses on the binary decisions (y_i), which determine whether a cell is published ($y_i = 0$) or suppressed ($y_i = 1$)
- Subproblems: verify if the current suppression pattern chosen by the Master Problem actually provides the required protection levels (lpl and upl) for each primary sensitive cell.

Original Problem

Given the original problem:

$$\begin{aligned} (P) = \quad & \min \quad c^T x + q^T y \\ & \text{s. to} \quad Tx + Wy = h \\ & \quad \quad x \in X \\ & \quad \quad y \geq 0 \end{aligned}$$

The two following problems are derivated:

• Master Formulation

$$\begin{aligned} (BP_r) = \quad & \min \quad z \\ & \text{s. to} \quad z \geq c^T x + u^{i\top}(h - Tx) \quad i \in I \subseteq \{1 \dots p\} \\ & \quad \quad v^{j\top}(h - Tx) \leq 0 \quad j \in J \subseteq \{1 \dots q\} \\ & \quad \quad x \in X. \end{aligned}$$

• Subproblem Formulation

$$\begin{aligned} (Q_D) = \quad & \max \quad u^T(h - Tx) \\ & \text{s. to} \quad W^T u \leq q \\ & \quad \quad u \in \mathbb{R}^m \end{aligned}$$

To take advantage of Benders decomposition we first reformulate the original MILP problem to an equivalent of Benders starting problem.

$$\begin{aligned}
\min \quad & \sum_{i=1}^n w_i y_i \\
\text{s. to} \quad & Ax^{l,p} = 0 & \forall p \in \mathcal{P} \\
& Ax^{u,p} = 0 & \forall p \in \mathcal{P} \\
& (l_i - a_i)y_i - x_i^{l,p} + s_{l,i}^{l,p} = 0 & i = 1 \dots n, \forall p \in \mathcal{P} \\
& (a_i - u_i)y_i + x_i^{l,p} + s_{u,i}^{l,p} = 0 & i = 1 \dots n, \forall p \in \mathcal{P} \\
& (l_i - a_i)y_i - x_i^{u,p} + s_{l,i}^{u,p} = 0 & i = 1 \dots n, \forall p \in \mathcal{P} \\
& (a_i - u_i)y_i + x_i^{u,p} + s_{u,i}^{u,p} = 0 & i = 1 \dots n, \forall p \in \mathcal{P} \\
& x_p^{l,p} + s_p^{l,p} = -lp l_p & \forall p \in \mathcal{P} \\
& x_p^{u,p} - s_p^{u,p} = up l_p & \forall p \in \mathcal{P} \\
& y_i \in \{0, 1\}, \quad x, s \geq 0
\end{aligned}$$

Given that y_i belongs to a restricted set and both x and s are only restricted to be positive, we take y_i as the x in Benders and $[x, s]$ as the y .

3 Datasets

TODO

4 Implementation Details

5 References

A AMPL Files

```
1 param n; # observations
2 param k; # clusters
3 set M:={1..n};
4
5 param d{M,M};
6 var x{M,M} binary;
7
8
9 minimize f: sum{i in M}
10      sum{j in M} d[i,j]*x[i,j];
11
12 subject to one_cluster{i in M}:
13      sum{j in M} x[i,j] = 1;
14
15 subject to k_clusters:
16      sum{j in M} x[j,j] = k;
17
18 subject to cluster_belongs{i in M, j in M}:
19      x[j,j] >= x[i,j];
```

Listing 1: kmedoid.mod

```
1 # Reset enviroment
2 reset;
3
4 # Load model and data
5 model kmedoids.mod;
6 data kmedoids.dat;
7
8 option solver cplex;
9 solve;
10
11 display _solve_elapsed_time;
12
13 printf {i in M, j in M} "%s,%s,%i\n", i, j, x[i,j] > "x_matrix.csv";
```

Listing 2: kmedoid.run

```
1 def generate_dat(N, D, k, filename='AMPL/kmedoid.dat'):
2     with open(filename, 'w') as f:
3         f.write("#Define sets\n")
4         f.write(f"param n := {N};\n")
5         f.write(f"param k := {k};\n")
6
7         f.write("#Define parameters\n")
8         f.write(f"param d: {'\t'.join([str(j+1) for j in range(N)])} :=\n")
9         for i in range(N):
10             f.write(f" {i+1} " + ' '.join(f"{D[i][j]:.6f}" for j in range(N)) +
11                 "\n")
12             f.write(";\n")
13
```

Listing 3: Python function to generate kmedoid.dat